Rapport de soutenance intermédiaire - Projet rPack

ByteShield

09/04/2025

Louis BEAUCAMPS
Arthur WAMBST
Marcus POIROT
Gabriel TONGBOONNAK

Contents

1 Introduction **3**
- 1.1 Qu'est ce qu'un packer ? . . . 3
- 1.2 Etat de l'art . . . 3
- 1.3 Notre projet . . . 5
- 1.4 L'équipe . . . 5

2 Avancée du Projet **6**
- 2.1 Introduction . . . 6
- 2.2 Rappel sur la première soutenance . . . 8
  - 2.2.1 White Box . . . 8
  - 2.2.2 Intégration dans notre projet en Rust . . . 9
- 2.3 Deuxième soutenance . . . 10
  - 2.3.1 Compression haute performance avec LZ4 . . . 10
  - 2.3.2 Choix du chiffrement AES-128 CBC . . . 11
  - 2.3.3 Protection Mémoire : Utilisation de `prctl` contre les Dumps Mémoire 12
  - 2.3.4 Vérifications temporelles . . . 12
  - 2.3.5 Anti-debug : Implémentation des fonctionnalités `ptrace` . . . 13
  - 2.3.6 Interface utilisateur . . . 15

3 Résultat vu dans un désassembleur **16**

4 Problèmes rencontrés **17**
- 4.1 Vérifications temporelles . . . 17
- 4.2 AES-128 ECB . . . 18
- 4.3 Contraintes liés à Rust pour le Développement Bas Niveau . . . 19
- 4.4 Migration vers Rust 2021 : Descente aux Enfers de Cargo . . . 21

5 Répartition des tâches **22**

6 Conclusion **22**

1 Introduction

1.1 Qu'est ce qu'un packer ?

Un **packer** est un outil d'encapsulation d'exécutables. Il sert à complexifier l'analyse statique et dynamique d'un binaire. L'objectif est de protéger le programme tout en garantissant son exécution normale et en limitant l'impact sur les performances de celui-ci. Cela passe par plusieurs procédés tel que la compression et le chiffrement de celui-ci, ainsi que l'implémentation de plusieurs méthode d'anti-débogage.

1.2 Etat de l'art

De très nombreux projets de packer existent et sont facilement trouvable en ligne (https://github.com/packing-box/awesome-executable-packing).

Exemple de UPX :

UPX (Ultimate Packer for eXecutables) est l'un des packers les plus connus. Il est utilisé pour compresser les fichiers exécutables sur plusieurs systèmes d'exploitation.

Il fonctionne de la manière suivante:

- L'exécutable est analysé pour identifier les sections (code, données, etc.). Ces sections sont compressées avec un algorithme comme LZ77.
- Un petit programme stub est ajouté à l'exécutable compressé. Ce stub est chargé de décompresser les données en mémoire lors de l'exécution.
- Lorsqu'on exécute l'exécutable packé, le stub charge le fichier compressé en mémoire, décompresse les données, puis exécute le code d'origine.

Après UPX, Themida est un autre packer très connu, mais il est utilisé principalement pour protéger les logiciels commerciaux.

Contrairement à UPX, qui est simple, gratuit et qui a pour objectif de compresser des données, Themida offre des fonctionnalités avancées comme:

- l'obfuscation
- la détection de débogueurs et d'outils de virtualisation
- la virtualisation du code

VMProtect, un packer principalement utilisé pour protéger des logiciels données critiques.

Ensuite, VMProtect est un packer avancé utilisé principalement pour protéger des logiciels contre la rétro-ingénierie, le piratage et le cracking.

La différence avec Themida est que VMProtect est utilisé pour des logiciels nécessitant une protection maximale, comme des programmes critiques ou des algorithmes confidentiels.

1.3 Notre projet

Nous avons pour ambition de développer un packer pour des exécutables ELF. Cette idée s'inscrit dans notre intérêt pour la cybersécurité, et plus particulièrement pour la rétro-ingénierie. C'est cette passion qui nous a motivés à concevoir un tel outil. Le choix du langage Rust s'avère particulièrement pertinent pour ce projet. En effet, Rust complique considérablement les tentatives de rétro-ingénierie grâce à son compilateur, qui produit un code machine optimisé et difficile à analyser. Cela s'explique notamment par des techniques telles que l'inlining agressif, la suppression des symboles inutiles, et la génération de noms et structures complexes. En somme, un packer écrit en Rust offre une protection robuste contre la rétro-ingénierie tout en maintenant de bonnes performances pour l'exécutable.

Pour voir les précédents rapports et surtout utiliser notre packer, consultez notre site.

1.4 L'équipe

Nous sommes un groupe de 4 étudiants d'EPITA :

Louis BEAUCAMPS (chef de projet)
Arthur WAMBST
Marcus POIROT
Gabriel TONGBOONNAK

2 Avancée du Projet

2.1 Introduction

Actuellement, nous avons implémenté les fonctionnalités suivantes, bien que seules la compression et l'AES soient intégrées a l'exécutable en sortie :

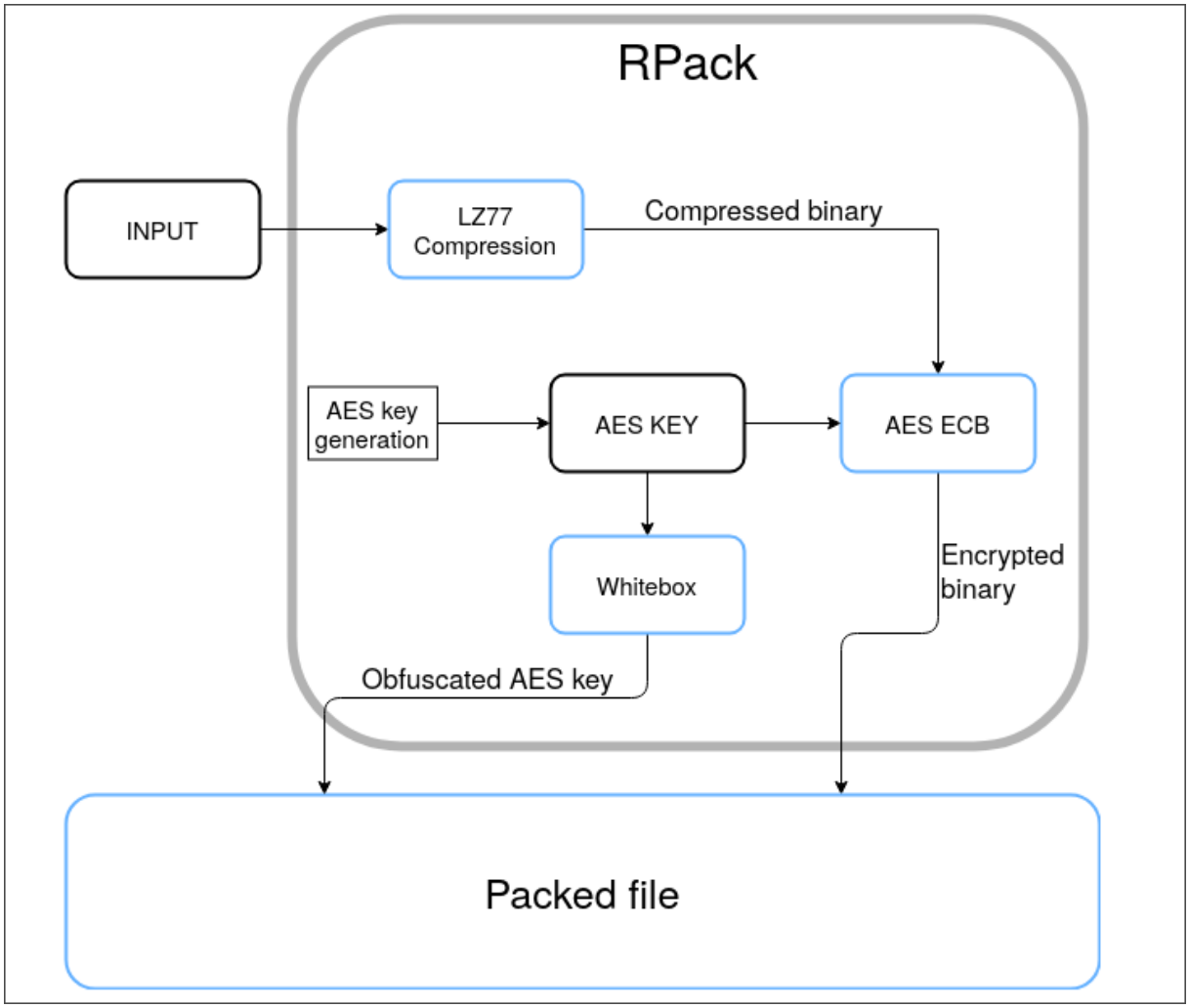

Figure 1: Packing process

Nous n'avons donc pas eu spécialement de problèmes majeurs dans la partie statique, en revanche la partie dynamique nous a forcé à fortement adapter le projet.

En effet, nous voulons que le fichier final packé ressemble a ceci, qui sera complété pour les prochaines soutenances par d'autres mecanismes :

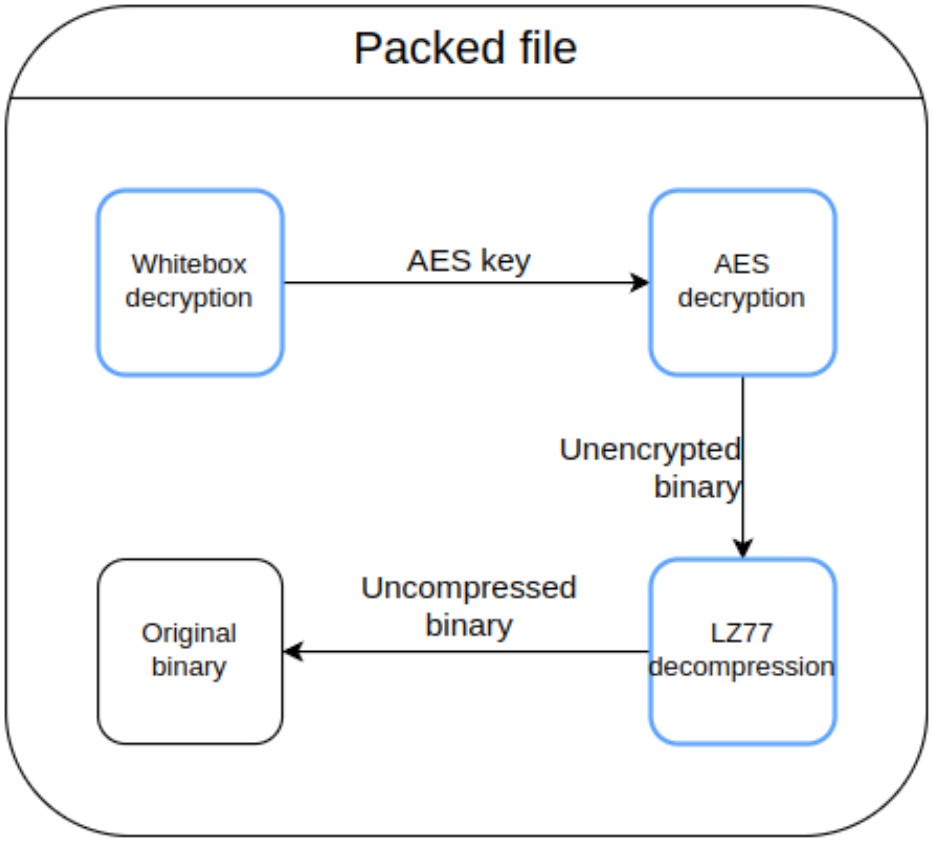

Figure 2: Packing process

2.2 Rappel sur la première soutenance

2.2.1 White Box

La White Box est une méthode conçue pour protéger les clés cryptographiques dans des environnements non sécurisés où l'attaquant a un accès complet au système (dans notre cas l'exécutable), cela inclue la mémoire et le code exécuté.
Contrairement à la cryptographie traditionnelle, où la clé est supposée rester secrète, une White Box intègre la clé directement dans les algorithmes de chiffrement, rendant son extraction particulièrement difficile. Ce mécanisme vise à résister à diverses attaques, notamment l'analyse statique, dynamique ou les attaques par canaux auxiliaires (side-channel attacks).
Dans le cadre de notre projet, nous protégeons la clé AES en l'intégrant dans une White Box.

Chiffrement (AES-128 ECB) : le chiffrement AES-128 ECB est un mode de chiffrement rapide et robuste pour le traitement de fichiers, car il traite chaque bloc de données indépendamment tout en utilisant une clé symétrique de 128 bits.
Cette clé assure un niveau élevé de sécurité en rendant difficile toute tentative de brute-force, tout en offrant une excellente rapidité et efficacité grâce à sa simplicité de mise en œuvre, ce qui le rend adapté aux situations nécessitant un bon compromis entre performance et sécurité.
Nous avons choisi d'utiliser une clé de 128 bits plutôt que 256 bits afin d'optimiser les performances.

Compression : nous utilisons une méthode de compression sans perte LZ77 qui est un algorithme fondamental toujours utilisé aujourd'hui. L'algorithme consiste à réduire au maximum les répétitions de données en cherchant des motifs en les remplaçant par des références à leurs occurrences précédentes.

2.2.2 Intégration dans notre projet en Rust

Nous avons opté pour une intégration de LZ4 via la bibliothèque `lz4_f` en Rust, qui propose une interface idiomatique et performante. Voici les aspects principaux que nous avons implémentés :

- **Encapsulation dans un module sécurisé** avec des types personnalisés (nouveauxtypes) pour les tampons compressés.
- **Interopérabilité C/Rust** pour les appels FFI (Foreign Function Interface), assurant la performance sans sacrifier la sécurité mémoire.
- **Gestion explicite des erreurs** via `Result<T, CompressionError>`, avec des conversions d'erreurs à la volée.
- **Support multithreading** pour paralléliser la compression de fichiers volumineux, à l'aide de rayon ou de threads natifs.

2.3 Deuxième soutenance

2.3.1 Compression haute performance avec LZ4

Contexte et origine Dans la suite logique de notre première implémentation basée sur LZ77, nous avons orienté notre projet vers l'algorithme **LZ4**, développé initialement par Yann Collet. Cet algorithme, publié en 2011, a été conçu pour répondre à un besoin industriel spécifique : offrir une compression ultra-rapide tout en maintenant une compatibilité avec les principes de LZ77.

LZ4 se distingue principalement par sa rapidité exceptionnelle et une simplicité structurelle qui permet une implémentation efficace sur des systèmes contraints ou haute performance.

Philosophie de LZ4 Contrairement à d'autres algorithmes de la même famille (DEFLATE, Zstd), LZ4 ne cherche pas à atteindre les meilleurs taux de compression, mais à garantir une vitesse de traitement maximale. Il vise à compresser les données plus vite que les disques ne les écrivent, ou plus vite que les réseaux ne les transmettent, ce qui en fait un excellent candidat pour les environnements à haute fréquence d'accès.

Structure interne de LZ4 LZ4 fonctionne selon une logique très simple de séquences successives. Chaque séquence est composée :

- **De littéraux** (non compressés) copiés directement dans la sortie.
- **D'une correspondance** (offset + longueur) vers une séquence déjà rencontrée.

Chaque séquence est encodée avec les éléments suivants :

1. Un **octet de token** où :
   - Les 4 bits de poids fort indiquent la longueur des littéraux.
   - Les 4 bits de poids faible indiquent la longueur du match.
2. Les **littéraux**, s'ils existent (0 à N).
3. Une **valeur d'offset** codée sur 2 octets (référence vers le début du match).
4. Un éventuel **champ de longueur supplémentaire** si la longueur dépasse 15.

Avantages techniques et performances

- **Vitesse de compression** : jusqu'à 400 Mo/s en compression simple-thread.
- **Vitesse de décompression** : jusqu'à 2 Go/s sur des architectures modernes.
- **Consommation CPU réduite** : idéal pour les systèmes avec contraintes processeur.
- **Faible overhead mémoire** : la structure compacte des métadonnées permet une excellente gestion des caches.
- **Mode streaming et blocs indépendants** : permet un traitement asynchrone et parallélisé.

2.3.2 Choix du chiffrement AES-128 CBC

Dans les premières versions du projet, nous utilisions le chiffrement **AES-128 en mode ECB** (Electronic Code Book) pour sa simplicité. Toutefois, ce choix a rapidement été abandonné au profit du **mode CBC** (Cipher Block Chaining), mieux adapté a nos demandes de sécurité et de performances.

Ce mode de chiffrement introduit une dépendance entre les blocs, renforcant sa robustesse en empêchant l'identification de motifs dans le fichier final. Deplus, un vecteur d'initialisation (IV) est utilisé pour sécuriser le chiffrement du premier bloc, assurant une bonne entropie dès le départ.

Nous conservons l'utilisation d'une **clé de 128 bits** afin de préserver un bon compromis entre sécurité et performance. AES-128 reste aujourd'hui l'un des standards les plus utilisés, offrant une résistance solide au bruteforce tout en restant très performant.

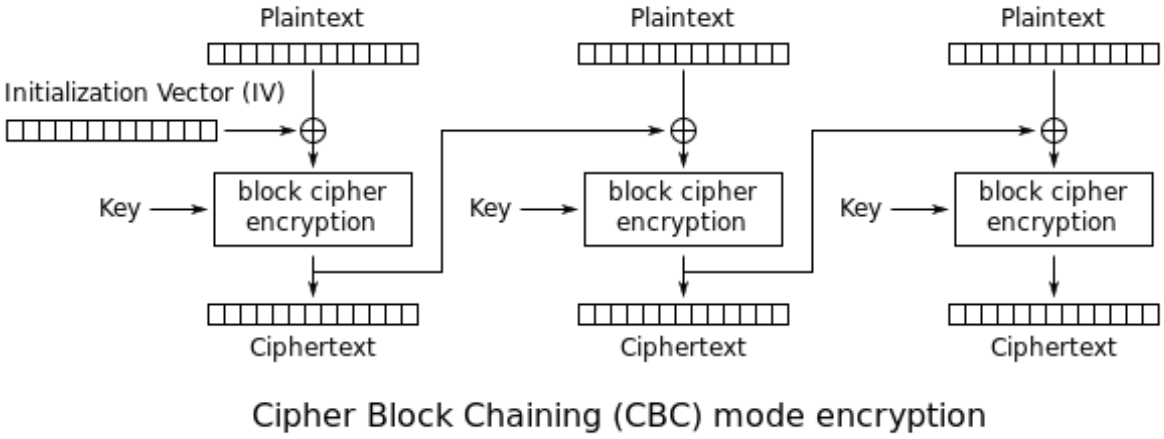

Illustration du chiffrement par blocs en mode CBC (Cipher Block Chaining)

2.3.3 Protection Mémoire : Utilisation de `prctl` contre les Dumps Mémoire

La fonction `prctl` (Process Control) est une interface mise à disposition par le noyau Linux qui permet de configurer divers aspects du comportement d'un processus.

Dans le contexte de l'anti-debugging et de la protection contre les dump de mémoire, l'option `PR_SET_DUMPABLE` est particulièrement utile, car elle permet de déterminer si un processus peut générer un fichier de vidage mémoire (coredump).

En désactivant cette option, on empêche le système de créer un dump mémoire lorsqu'un processus se termine de manière anormale, ce qui pourrait être exploité par un attaquant pour récupérer des informations sensibles en mémoire, comme des clés cryptographiques.

Étant donné que nous n'avions pas encore beaucoup de connaissances en Rust, nous pensions que le langage ne disposait pas directement d'une API native pour `prctl`, et que nous devions ré-implémenter cette fonctionnalité.

Cependant, après avoir fait plus de recherches a ce sujet, nous avons découvert que prctl est déjà implémentée en Rust. Cela nous permet de contrôler le comportement du processus au niveau du noyau et d'ajouter cette couche de sécurité contre les tentatives de dump mémoire, tout en restant cohérent avec les principes de sécurité et de performance de notre application.

2.3.4 Vérifications temporelles

Les vérifications temporelles permettent de détecter la présence de breakpoints.
En effet, les breakpoints provoquent l'interruption de l'exécution du programme par le débogueur, permettant ainsi d'inspecter les registres à ce moment-là.

En analysant le temps d'exécution de certaines instructions à des endroits stratégiques où l'attaquant est susceptible de placer un breakpoint, on peut détecter ces interruptions, car elles introduisent un délai supplémentaire par rapport à l'exécution normale du programme.

Dans notre implémentation, nous allons mesurer précisément le temps d'exécution de plusieurs sections du programme, en particulier autour des points où des breakpoints sont les plus susceptibles d'être placés. Si une différence de temps est détectée, cela indiquera potentiellement l'utilisation d'un débogueur.

Pour cela nous avons donc utilise la librairie std::time de Rust, ce qui nous permet tout simplement de mesurer l'heure a un point A, puis plus tard a un point B.

Il ne reste qu'a faire la différence entre ces 2 moments et les comparer a un certain seuil en fonction de ce que nous attendons.

2.3.5 Anti-debug : Implémentation des fonctionnalités `ptrace`

La méthode `ptrace` est une technique courante utilisée pour la détection et la prévention du débogage dynamique. Sous les systèmes Unix, `ptrace` permet à un processus d'attacher un débogueur à un autre processus en cours d'exécution. Cela permet au débogueur d'inspecter, de modifier les registres, et d'interrompre l'exécution du programme cible, entre autres actions. C'est la méthode principale utilisée par des outils comme `gdb` pour analyser un programme en cours d'exécution.

Le fonctionnement de `ptrace` est simple : si un processus appelle `ptrace` pour se tracer lui-même (via `PTRACE_TRACEME`), il permet à un débogueur d'attacher et de suivre son exécution. Toutefois, si `ptrace` échoue (en renvoyant une valeur négative), cela signifie qu'un débogueur est déjà attaché au programme, ce qui peut signaler une tentative de débogage ou d'analyse dynamique.

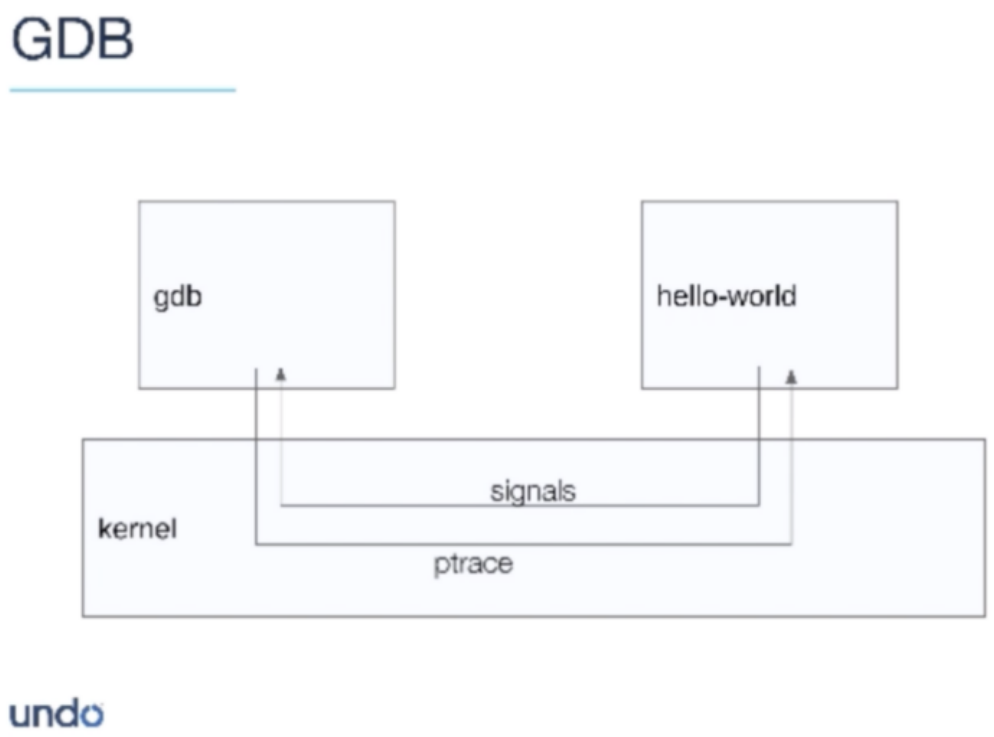

Figure 3: Utilisation de ptrace par gdb

Dans le cadre de notre projet, nous cherchons à implémenter une défense contre l'analyse dynamique en utilisant cette méthode pour détecter toute tentative de débogage. En appelant `ptrace` dans notre programme, nous pouvons vérifier si l'appel réussit ou échoue. Si l'appel échoue (ce qui indiquerait qu'un débogueur a tenté de s'attacher au processus), nous pouvons immédiatement interrompre l'exécution du programme pour éviter toute tentative d'exploitation par un attaquant.

Étant donné que notre projet est développé en Rust, nous avons ré-implémenté cette fonctionnalité afin de pouvoir bloquer toute tentative de débogage. Cela permet de renforcer la sécurité de notre application en rendant l'analyse dynamique beaucoup plus difficile, car l'attaquant devra patcher le binaire directement avant de pouvoir utiliser un debugger.

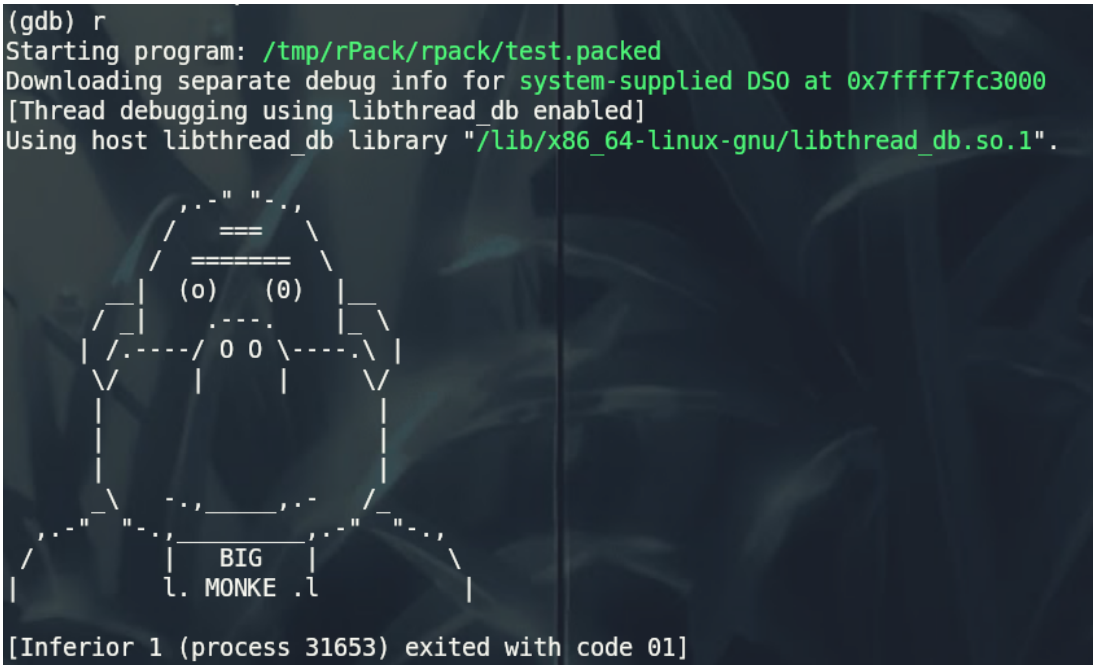

Figure 4: Utilisation de gdb bloquée par notre packer

2.3.6 Interface utilisateur

A la 1ère soutenance, nous n'avions pas encore pu tout combiner dans un seul exécutable, notamment car nous avons étés retardes par des problèmes lors de l'execution en memoire (cf. 1ere soutenance) et d'i/o.

Ceux-ci étant maintenant réglés, nous avons alors pu compiler notre exécutable et l'intégrer dans un site web fini, dont voici le lien : https://rpack.la-banquise.fr/

Ce site web contient nos rapports de soutenances, le lien vers le wiki que nous maintenons depuis le début du projet et surtout permettre de packer un fichier en ligne.

Par contre, il nous est impossible d'ajouter des éléments pour améliorer l'expérience utilisateur dans le fichier packe (on nous a par exemple demander d ajouter des éléments / print du texte lors de l'exécution), car cela ne ferait que fournir a un attaquant a avoir un point d'entrée. Nous avons donc seulement rajouté un print lorsque nous détectons l'utilisation d'un debugger.

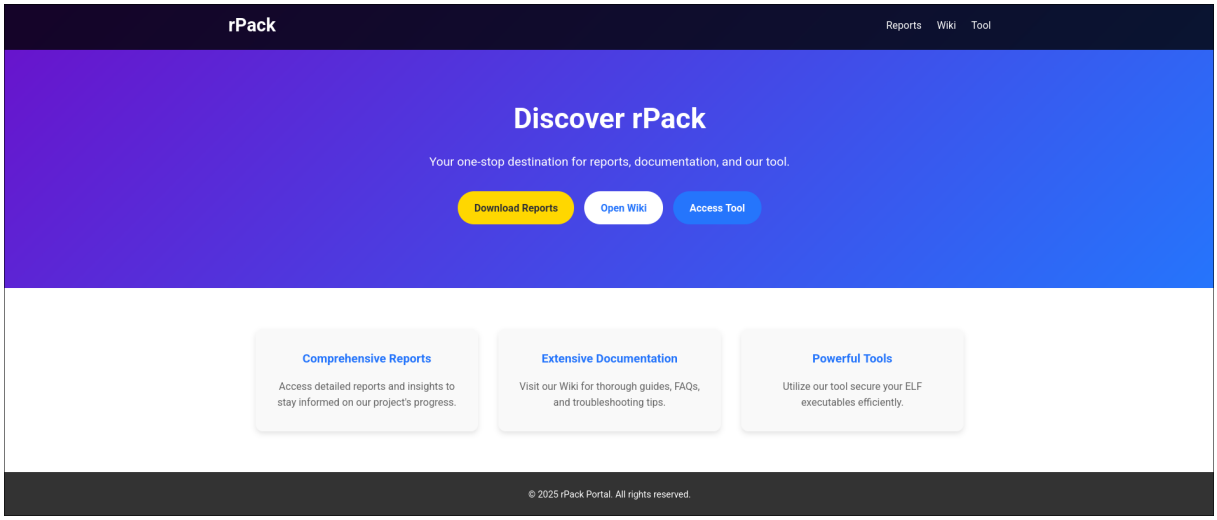

Figure 5: rPack website

3 Résultat vu dans un désassembleur

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  printf("Hello World!");
  return 0;
}
```

```
}
sub_27880(&v41);
if ( __OFSUB__(-v41, 1LL) )
{
  *(_QWORD *)fd = v42;
  sub_8630(aFailedToGetCur, 37LL, fd, &off_5E628, &off_5E7F8);
}
v5 = v42;
v41 = 0x1B600000000LL;
LODWORD(v42) = 1;
WORD2(v42) = 0;
sub_27F10(fd, &v41, v5, v43);
if ( *(_DWORD *)fd == 1 )
{
  v41 = *(_QWORD *)&fd[8];
  sub_8630(aFailedToOpenCu, 33LL, &v41, &off_5E628, &off_5E810);
}
v26 = *(_DWORD *)&fd[4];
fcntl(*(int *)&fd[4], 2, 1LL);
sub_27A60(&v41, &v26);
if ( (_DWORD)v41 == 2 )
{
  *(_QWORD *)fd = v42;
  sub_8630(aFailedToGetFil, 27LL, fd, &off_5E628, &off_5E828);
}
v6 = v45;
v7 = sub_27ED0(&v26, 0LL, v45 - 8);
sub_8A30(v7, v8, &off_5E840);
v34[0] = 0LL;
v9 = sub_A640(&v26, v34, 8LL);
sub_8A90(v9, aFailedToReadDe, 32LL, &off_5E858);
v10 = v34[0];
v6 -= 16LL;
v11 = sub_27ED0(&v26, 0LL, v6);
sub_8A30(v11, v12, &off_5E870);
v13 = sub_A640(&v26, v34, 8LL);
sub_8A90(v13, aFailedToReadEn, 29LL, &off_5E888);
v14 = v34[0];
v15 = sub_27ED0(&v26, 0LL, v6 - v34[0]);
sub_8A30(v15, v16, &off_5E8A0);
sub_8AE0(v46, v14);
v17 = v47;
v18 = v48;
v19 = sub_A640(&v26, v47, v48);
sub_8A90(v19, aFailedToReadEn_0, 32LL, &off_5E8D0);
```

Avant/apres packing (code)

Functions

Function name

- _init_proc
- sub_1020
- sub_1030
- **_cxa_finalize**
- **_printf**
- _start
- deregister_tm_clones
- register_tm_clones
- __do_global_dtors_aux
- frame_dummy
- **main**
- _term_proc
- **__libc_start_main**
- **printf**
- **__imp__cxa_finalize**
- __gmon_start__

Functions

Function name

- sub_39610
- sub_39A40
- sub_3A0D0
- sub_3A490
- sub_3A550
- sub_3A6D0
- sub_3A800
- sub_3AEC0
- sub_3B6E0
- sub_3B7A0
- sub_3B810
- sub_3B8A0
- sub_3B900
- sub_3B9B0
- sub_3B9D0
- sub_3BA20
- sub_3BC00
- sub_3BDE1
- sub_3BE45
- sub_3BE65
- sub_3BF8F
- sub_3C119
- sub_3C199
- sub_3C21C
- sub_3C2CF
- sub_3C39B
- sub_3C439

Avant/apres packing (ida)

4 Problèmes rencontrés

4.1 Vérifications temporelles

La mise en place de vérifications temporelles (timechecks) a été conceptuellement simple, mais délicate à affiner. L'idée était de mesurer la durée d'exécution de certaines instructions critiques pour détecter la pause induite par un breakpoint.

Cependant, en pratique, plusieurs problèmes sont apparus :

- **Variabilité naturelle du temps d'exécution** selon la machine, la charge CPU ou le contexte d'exécution.
- **Difficulté à définir un seuil fiable** entre un ralentissement normal et une pause liée à un breakpoint.

Nous avons dû effectuer de nombreux tests sur différents systèmes pour calibrer les seuils temporels et éviter les faux positifs. La robustesse de cette méthode reste donc dépendante de l'environnement d'exécution, mais elle représente malgré tout une barrière supplémentaire contre le debugging.

4.2 AES-128 ECB

Initialement, le choix d'utiliser le mode ECB venait de sa simplicité. Cependant, après reconsidérations ce choix s'est révélé problématique, voire dangereux pour la sécurité de notre packer. Deplus il n'était pas suffisament performant et mettait environ 4 secondes pour chiffrer un fichier de quelques méga octets sur un ordinateur moderne.

Le mode ECB chiffre chaque bloc de données indépendamment. Donc deux blocs identiques produiront le même résultat. Cela crée des motifs permettant à un attaquant d'identifier des structures répétitives.

A l'inverse le mode CBC introduit une dépendance entre les blocs, ce qui masque efficacement les motifs et donc renforce la sécurité.

Illustration du limitations du chiffrement par blocs en mode ECB (Electronic Code Book)

4.3 Contraintes liés à Rust pour le Développement Bas Niveau

Le choix du langage **Rust** pour rPack a été motivé par ses qualités en termes de **sécurité mémoire, de performance et de sûreté**, et par le fait que certaines de ses caractéristiques, comme la monomorphisation, peuvent rendre la rétro-ingénierie des binaires produits plus ardue.

Rust est réputé pour son système de typage rigoureux, son ownership model et son borrow checker, qui permettent de prévenir de nombreuses erreurs de programmation, en particulier celles liées à la gestion de la mémoire. Cependant, notre projet nécessite de manipuler la mémoire et de produire du code bas niveau, ce qui confronte Rust à ses limites et impose des compromis, notamment en ce qui concerne la sécurité et l'utilisation du code `unsafe` qui est utilisé sur la plupart des fonctions.

En effet, l'essence même du packer réside dans la **manipulation directe de la mémoire et des appels système**. Les crates `encore`, `stage1` et `stage2` interagissent intimement avec le kernel pour réaliser des opérations fondamentales : allocation et gestion de la mémoire virtuelle (`mmap`, `munmap`), lecture et écriture de fichiers, exécution de code arbitraire en mémoire, etc. Ces opérations, par nature, **échappent au modèle de sécurité de Rust** et nécessitent l'utilisation de blocs de code `unsafe`.

L'utilisation de code `unsafe` est ainsi **omniprésente** dans les crates de bas niveau du projet.

On la retrouve notamment pour :

- **Les points d'entrée (`entry`, `_start`)** : Ces fonctions, qui constituent les points de départ de l'exécution, sont nécessairement déclarées comme `unsafe` et implémentées en assembleur inline pour interagir directement avec le chargeur et le système d'exploitation au niveau le plus bas.
- **Les appels système (syscalls)** : Le crate `encore` encapsule les appels système Linux standards en utilisant de l'assembleur inline et des fonctions `unsafe`, car ces interactions directes avec le noyau sont intrinsèquement non sûres.
- **Le mapping mémoire (mmap)** : La gestion de la mémoire virtuelle, cruciale pour le chargement et la relocation des exécutables, repose sur l'appel système `mmap` et implique donc l'utilisation de code `unsafe` pour manipuler directement les adresses mémoire.
- **L'analyse et la manipulation des structures ELF** : Le crate `pixie` doit analyser et manipuler des structures de données binaires complexes (headers ELF, segments, tables de symboles), ce qui requiert souvent l'interprétation de données brutes en mémoire et l'utilisation de raw pointers, nécessitant des blocs `unsafe`.

Face à cette omniprésence du code `unsafe`, l'approche adoptée dans rPack a été de **contenir et de maîtriser au maximum son utilisation**.

L'objectif n'est pas de supprimer le code `unsafe` (ce qui serait impossible pour un projet de cette nature), mais de le **restreindre à des modules spécifiques et bien isolés** (comme `encore` et certaines parties de `pixie`) et de construire des **abstractions sûres** au-dessus de ces primitives non sûres.
L'API fournie par le crate `encore`, par exemple, vise à offrir une interface Rust conviviale et sécurisée pour les opérations système, même si son implémentation interne repose sur du code `unsafe`.

En conclusion, le développement de rPack en Rust représente un **compromis constant entre la sécurité et la nécessité d'opérations bas niveau**.

Si le langage Rust apporte des garanties précieuses en termes de sûreté et de performance, la nature même du projet impose l'utilisation de code `unsafe` pour interagir avec le système d'exploitation et manipuler directement la mémoire.
L'effort principal réside alors dans la **gestion rigoureuse** de ce code `unsafe`, en le confinant et en construisant des abstractions sûres pour minimiser les risques et préserver autant que possible les avantages de Rust.

Globalement, le Rust est un choix pertinent pour les applications *userland* grâce à ses garanties de sécurité et de performance. Cependant, le développement bas niveau nécessitant de manipuler la mémoire, dont notre *packer*, nécessite un usage important de code `unsafe`, ce qui, bien que nécessaire dans ce contexte, va à l'encontre des principes fondamentaux du language.

4.4 Migration vers Rust 2021 : Descente aux Enfers de Cargo

L'exécution en mémoire de notre exécutable suit un article de fasterthanlime (https://fasterthanli.me/series/making-our-own-executable-packer) datant de quatre ans, rédigé pour l'édition Rust 2018 avec la toolchain "nightly-2021-02-14".
Notre code étant développé sous l'édition 2021, des problèmes de compatibilité se sont manifestés. Nous avons privilégié le portage du code de l'article vers l'édition 2021, plutôt que d'adapter notre projet à l'édition 2018. Malheureusement, cette migration s'est avérée complexe et ardue, soulignant les défis de l'écosystème Rust et de Cargo.

L'article original repose sur des modules qui ne sont plus maintenues, notamment à `linked_list_allocator` qui dépend de `lock_api` qui n'est plus maintenue. Cela a engendré des problèmes de dépendances implicites obsolètes, souvent compliquées à identifier à cause des messages d'erreur de Cargo. On se retrouve contraints d'isoler et de tâtonner afin d'identifier le module qui utilise la dépendance. En effet, le compilateur Rust, bien que verbeux, produit souvent des messages d'erreur peu informatifs

Ainsi, la tentative de porter le projet vers l'édition 2021 a rapidement révélé, par le biais de Cargo, des problèmes de compatibilité et des régressions inattendues liés à cette transition d'édition.

La modernisation du projet s'est transformée en une véritable **chasse aux dépendances implicites**. De nombreuses bibliothèques ou fonctionnalités du langage, qui étaient disponibles ou se comportaient d'une certaine manière en 2018, ont subi des changements subtils mais significatifs dans l'édition 2021. Lorsque l'on réglait un problème 15 autres apparaissaient et actuellement, le projet se débat encore avec des **erreurs de linkage complexes**.

Cette tentative de portage a permis de révéler une facette moins reluisante de l'écosystème Rust. Si Cargo facilite grandement la gestion des dépendances dans de nombreux cas, il peut aussi se montrer impuissant face à des **changements profonds et non rétrocompatibles entre les éditions du langage**. On peut également mentionner crates.io, qui ne fournit pas l'information sur la version de Rust requise par chaque version d'une librairie. Ce qui nous contraints à procéder par tâtonnements, engendrant une perte de temps significative.

Après de nombreuses tentatives, nous avons abandonné l'idée de porter minipak vers l'edition 2021 et avons décidé d'utiliser memfd plutot que de porter l'executable vers une architecture custom puis de l'executer dans une VM.

5 Répartition des tâches

Tâches	Louis	Arthur	Marcus	Gabriel
Compression		Suppléant		Délégué
Chiffrement			Délégué	Suppléant
Whitebox	Délégué		Suppléant	
Site Web		Délégué		Suppléant
Protection Mémoire	Suppléant			Délégué
Anti-débogage	Suppléant	Délégué		
Time Check		Suppléant	Délégué	

Répartition des tâches entre les membres de l'équipe

6 Conclusion

Depuis la première soutenance, notre projet a significativement évolué. Après avoir consolidé les bases (compression LZ4, chiffrement AES-128 CBC et la whitebox), nous avons mis l'accent sur le renforcement de la sécurité dynamique de notre packer tout en unifiant les travaux effectués avant première soutenance.

Nous avons intégré trois mécanismes majeurs d'anti-debugging :

- L'utilisation de `prctl` pour désactiver les dumps mémoire, renforçant la protection des données sensibles à l'exécution.
- L'implémentation de `ptrace` pour détecter et bloquer les tentatives de débogage via des outils comme `gdb`.
- L'ajout de vérifications temporelles afin de repérer les pauses anormales dans l'exécution du programme, pouvant indiquer la pose de breakpoints.

Ces ajouts marquent une étape importante vers un packer robuste, difficile à analyser aussi bien statiquement que dynamiquement. Cela dit, leur intégration n'a pas été sans difficultés, notamment en raison des contraintes du langage Rust pour manipuler directement la mémoire et les appels système.

Nous avons également pris le temps d'explorer plus en profondeur les limites de Rust dans un contexte bas niveau, ce qui nous a permis d'optimiser notre gestion du code `unsafe`, tout en maintenant des abstractions sûres dans les couches supérieures du projet.

Pour la troisième soutenance, nous prévoyons d'aller plus loin dans la protection dynamique du binaire, avec l'implémentation de :

- **L'obfuscation des appels système**, potentiellement à l'aide de techniques basées sur `eBPF`, pour compliquer davantage l'analyse dynamique.
- **Des checksums sur des composants critiques** comme `crt0` et les paramètres de la whitebox, pour détecter toute tentative de modification du binaire.
- **Un driver d'analyse personnalisé**, basé sur QEMU ou GDB, pour automatiser nos tests d'injection, d'analyse et de contournement des protections.

Ce projet, bien qu'exigeant techniquement, reste pour nous une aventure passionnante. Nous restons motivés à le mener jusqu'au bout, avec pour objectif final un packer complet, stable, et difficile à contourner.